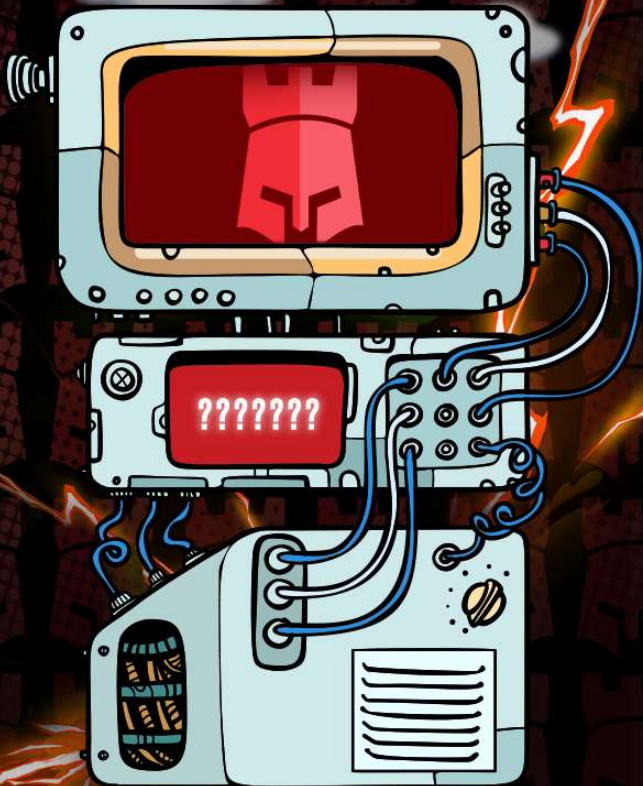


THAT **SHOULDN'T** HAVE WORKED:

PAYLOAD DEVELOPMENT 101



WHO IS **COREY OVERSTREET**

Senior Security Consultant

- 14 Years Experience
 - Penetration Testing
 - Red Teaming
 - IT and Security Expertise
 - Systems Admin
- OSEP, OSCE, OSCP, OSEE, CRTO
- Musician and Tinkerer



The **Who** and the **Why**

Who is this for?

- (Internal) Red Teamers
- Blue Teamers

Why?

- Everything is stealthy until they're/you're watching for it

The Cat and Mouse Game

- What works today might not work tomorrow
- Keep it simple
- Every environment is different
- Nothing here is new or groundbreaking but could be new to you or your team

Researchers

Open Source
Tool Writers

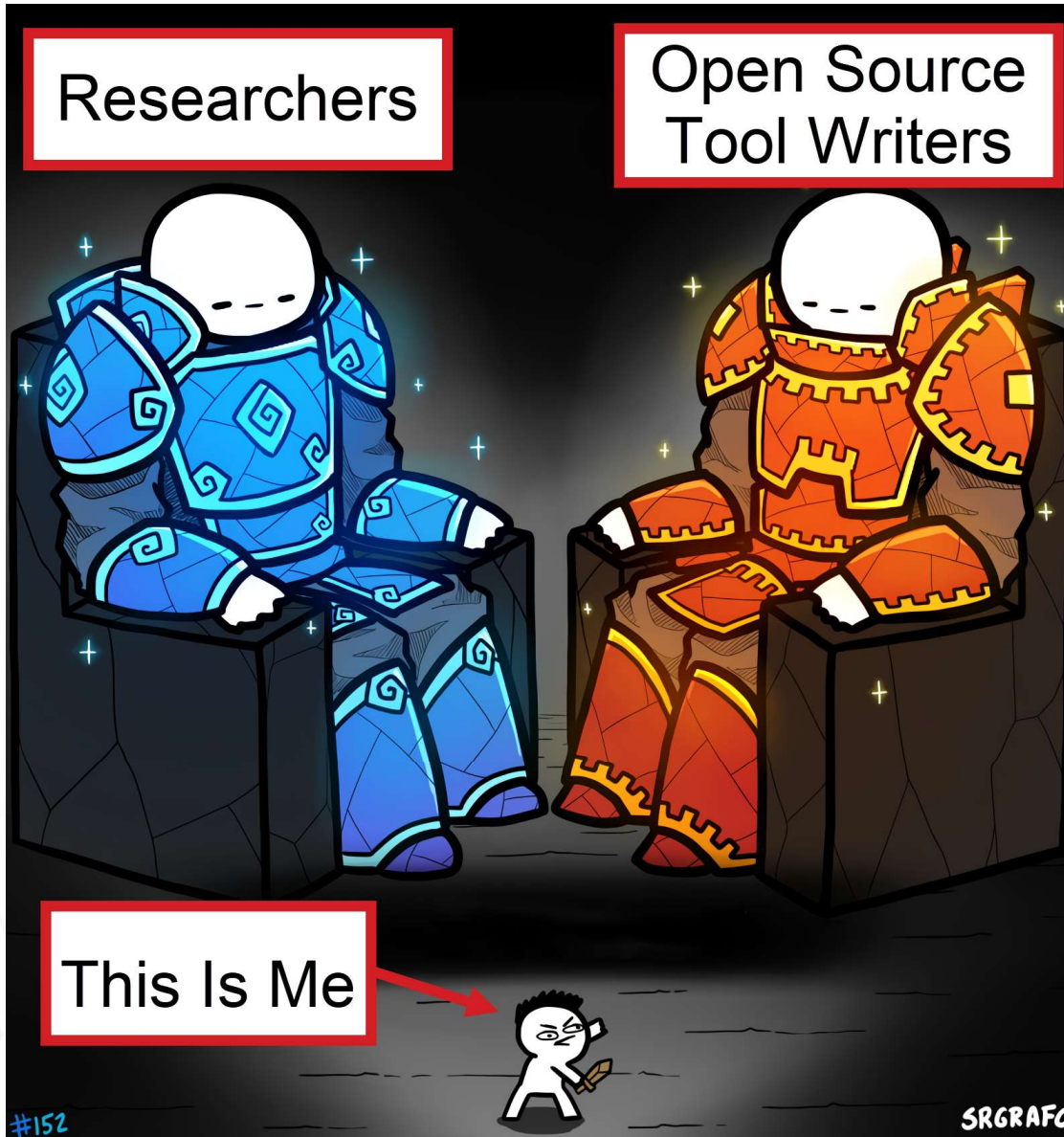
This Is Me

REDSIEGE.COM

#152

SRGRAFO

REDSIEGE
INFORMATION SECURITY



Overview

- Initial Considerations
- Bypassing Initial Protections and Detections
- Getting Out of the Sandbox
- Static Detections
- Our Most Common Execution Methods
- Resources



Initial Considerations

Programming Languages

- C/C++
- .NET
 - PowerShell
 - C#
 - F#
- VBA
- Rust
- Golang
- Nim
- Python

Common File Formats

- Portable Executable (EXE)
- Dynamic Link Library (DLL)
- XML (.NET code payloads)
- Microsoft Installer Files (MSI)
- Microsoft Word/Excel Add-in (WLL/XLL)
- HTML Applications (HTA)
- ClickOnce Applications (Application)
- Script Files (BAT/CMD)
- Jscript (JS)

(Simulated) Delivery

- Executing payloads generated locally VS. downloaded may not be detected or blocked the same way
- Download through different means
 - Browser
 - PowerShell
 - Bitsadmin
 - Curl
- Zone.Identifier
 - a.k.a. Mark of the Web (MotW)
 - Stored as Alternate Data Stream in file
 - Get-Item <payload file> -Stream zone*

Zone.Identifier Bypasses

- Droppers
 - Base64 Encoded File
 - AES Encrypted
- Containers
 - ZIP Files
 - TAR Archives
 - Virtual Hard Disks (VHD/WIM)
 - ISO Files (Deprecated)

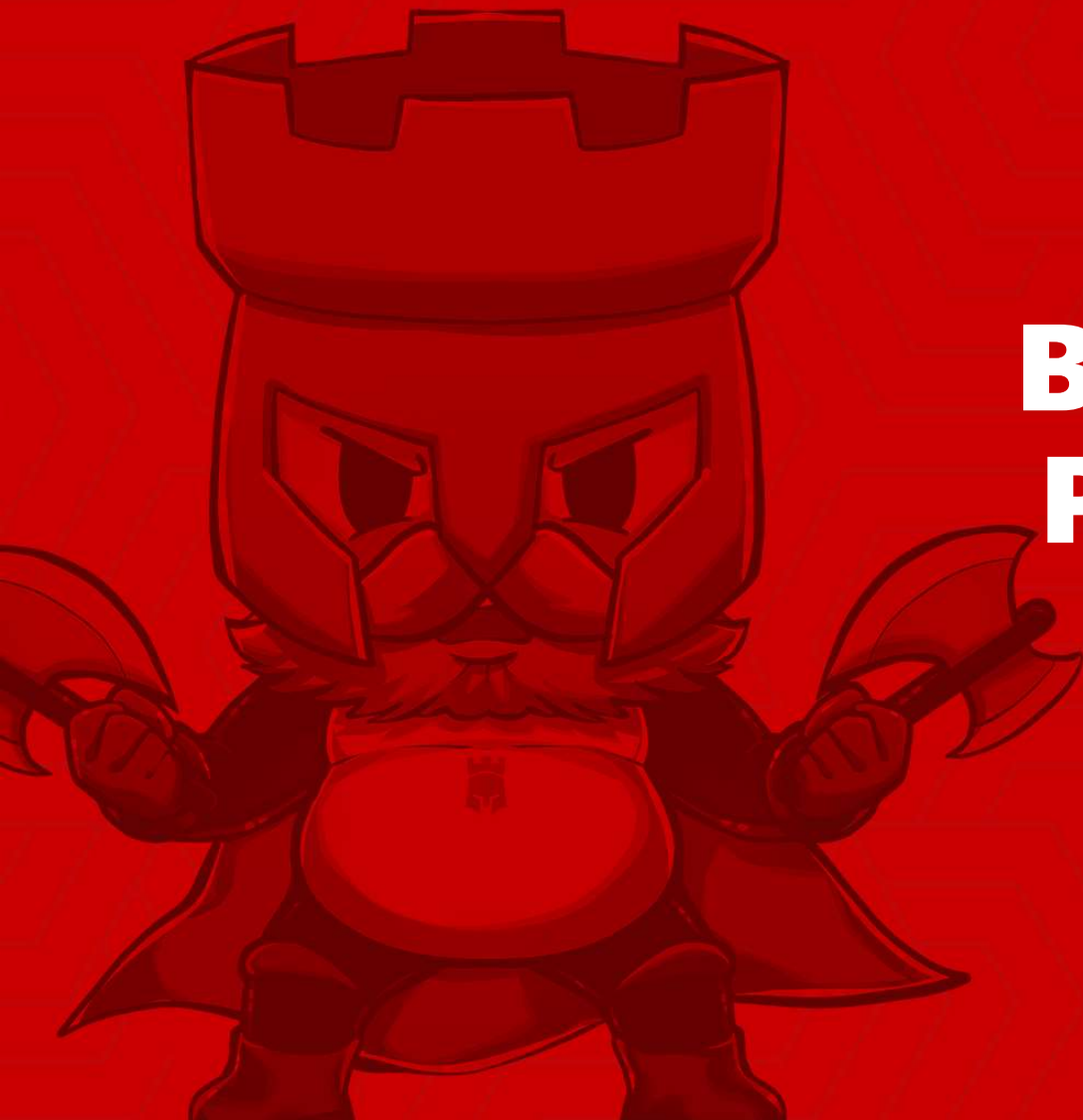
(Simulated) Delivery

```
PS C:\Users\test\downloads> Get-Item .\test.exe -Stream zone*
```

```
PSPath      : Microsoft.PowerShell.Core\FileSystem::C:\Users\test\downloads\test.exe:Zone.Identifier
PSParentPath : Microsoft.PowerShell.Core\FileSystem::C:\Users\test\downloads
PSChildName  : test.exe:Zone.Identifier
PSDrive      : C
PSProvider    : Microsoft.PowerShell.Core\FileSystem
PSIsContainer : False
FileName     : C:\Users\test\downloads\test.exe
Stream       : Zone.Identifier
Length       : 84
```

```
PS C:\Users\test\downloads> tar -cf test.tar .\test.exe
PS C:\Users\test\downloads> tar -xf .\test.tar
PS C:\Users\test\downloads> Get-Item .\test.exe -Stream zone*
PS C:\Users\test\downloads> |
```

No Zone.Identifier



Bypassing Initial Protections and Detections

AMSI

- Anti-Malware Scanning Interface
- Loaded into every process through `amsi.dll`
- Used by Microsoft Defender
- Loaded in Userland
- Can be bypassed in user process but bypasses are also being signed

AMSI Bypasses

- Modifying Signed Strings
- Zero size AmsiScanBuffer Patch
- Patching amsiInitFailed() to return AMSI didn't load
- Use PowerShell version 2 – No AMSI integration
- AMSI.fail – randomized AMSI bypasses

ETW

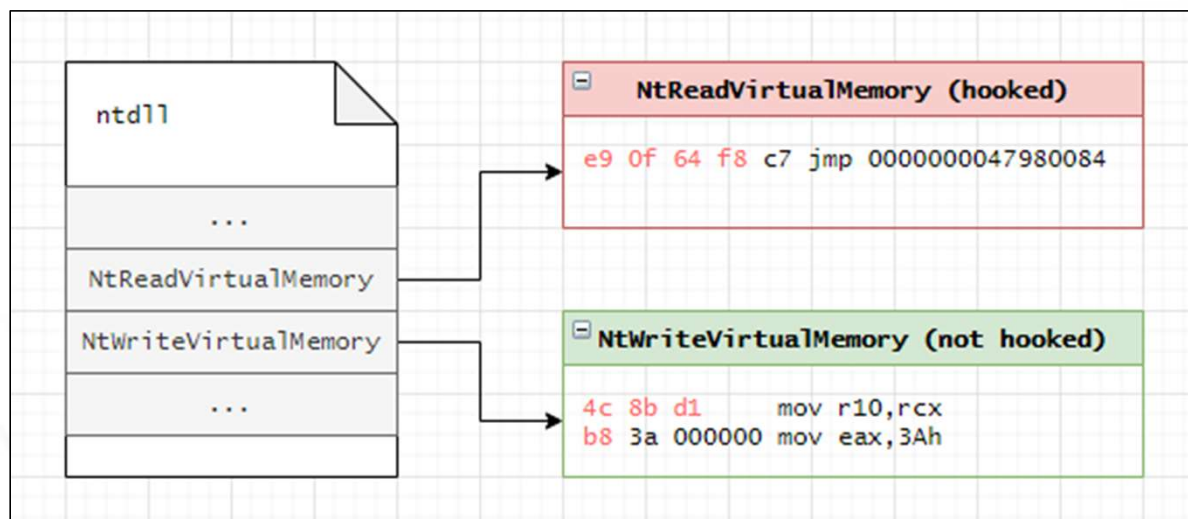
- Event Tracing for Windows
- EDR products often have custom consumers
- API calls are contained in ntdll.dll
- Again, can be bypassed but detection of tampering can be detected

ETW Bypasses

- Obfuscating .NET Tooling
 - Change known strings (mimikatz > mimidogz)
 - Change signed namespaces (Rubeus)
 - Change default title, product, GUID, etc.
- Patch ETW API call hooks
 - EDR relies on ETW as a source for monitoring endpoint actions

EDR Hooks

- Windows API calls are patched to divert code execution to EDR DLL codespace



EDR Unhooking

- Parse each Windows API call in loaded DLLs
 - Overwrite with stock bytes
- Overwrite entire DLL in process memory with clean copy from disk
- Freeze method
 - Newly created suspended process has DLLs in original state
- Chicken and the Egg problem

Syscalls

- Nt* or Zw* API calls are proxy to Kernel functions
- Identified with System Service Number (SSN)
- Can be stealthier than userland API calls

```
0:004> uf ntdll!NtWriteFile
ntdll!NtWriteFile:
00007ffc`5f55f850 4c8bd1      mov     r10,rcx
00007ffc`5f55f853 b808000000    mov     eax,8 ← NtWriteFile Syscall nr: 0x08
00007ffc`5f55f858 f604250803fe7f01 test    byte ptr [SharedUserData+0x308 (00000000`7ffe0308)],1
00007ffc`5f55f860 7503        jne     ntdll!NtWriteFile+0x15 (00007ffc`5f55f865) Branch

ntdll!NtWriteFile+0x12:
00007ffc`5f55f862 0f05        syscall ← syscall CPU instruction
00007ffc`5f55f864 c3          ret

ntdll!NtWriteFile+0x15:
00007ffc`5f55f865 cd2e        int     2Eh
00007ffc`5f55f867 c3          ret
```

How To **Generate Syscalls**

- Syswhispers 2/3
- Sektor7 HellsGate
- Sektor7 HalosGate
- Unmanaged ASM Code in C#



Getting Out of the Sandbox

Where's The **Sandbox**?

- Antivirus and EDR will execute files in a sandbox to watch for suspicious behavior
- Executed from small virtual machine locally or remotely by endpoint protection vendor
- Telltale signs can be determined programmatically prior to payload detonation

Sandbox Bypass: **Time**

- Name of the game: Slow down execution
- Windows API Sleep calls
 - Can be simulated by EDR sandbox
 - Sleep API calls can be detected
- Calculate large prime numbers
- Calculate Fibonacci Sequence
- Endpoint protections can not hinder/slow down user experience

Sandbox Bypass: **Size**

- Name of the game: Be too big to fit
- Pad out with zeros or junk data
 - DigDug
 - Mangle
- Endpoint protection will not scan large files to avoid slowdowns



Sandbox Bypass: **Domain Status**

- Name of the game: Check if we're in the target environment
- Check Domain-Joined status
- Check for proper domain name suffix
- Sandbox VM will rarely pass this test
- Prior knowledge of internal domain names can be a downside

Sandbox Bypass: **Host Attributes**

- Monitor Resolution
 - Amount of RAM
 - MAC Address
 - Number of processor cores
 - Executing username
 - Debugging processes attached
- Code examples in a multitude of languages can be found in the Github repo CheckPlease

■ C#	Restructured folders
■ C	Restructured folders
■ Go	Create checkplease.go
■ Multi-Language Support/Delay-Analy...	Restructured folders
■ Perl	Restructured folders
■ PowerShell	Restructured folders
■ Python	Fix typo in registry_size.py
■ Ruby	Restructured folders
■ Rust	file name hash and mac address checks
■ vbs	Quit if true



Static Detections

Embedded Shellcode

- Most of the time, we are trying to load position independent binary shellcode
- Embedded shellcode stored in program variables can often be detected at rest
 - Signed byte patterns
 - Large data sections with high entropy
- Encryption/encoding can increase entropy significantly

Bypassing Shellcode **Detection**

- Store the data using known or common data types
 - Universal Unique Identifiers (UUIDs)
 - Embedded in image files (JPG, PNG, GIF)
- Map the bytes to lower entropy data
- Jargon
 - Maps each of the possible 255 bytes to a dictionary word
 - 0x01 = "cat" , 0x02 = "ball", 0x03 = "sky", etc.
 - Decode at runtime in memory
- Use a different encryption key every time

Static Signatures

- Different compilers optimize code in different but predictable ways
- If it came from Github, it is signed
- If it is a widely used C2 framework, the default payloads are signed
 - Cobalt Strike
 - Sliver
 - Covenant

Static **Signature** Bypasses

- Bypass public code signatures
 - Change variable names and/or lengths
 - Change the checksum or hash of the final payload
 - Change the default encryption keys
- Bypass default C2 framework detections
 - Modify default Cobalt Strike tooling with kits
 - Recompiling CS kits without any code changes can often bypass static detections

Order of Operations

- Most malware follows the same order of operations
 - Provision memory space
 - Copy payload into memory
 - Make executable
 - Run
- Break up the order with unnecessary code
 - Implement needless branches and dead-end code to break signatures

Don't Be **Suspicious** Immediately

- Immediate C2 callback after execution
- Parent-Child process tree
 - Why is Microsoft Word executing cmd.exe?
- Injecting into processes that shouldn't talk to the internet
 - Notepad is now a web browser!
- Memory Protections
 - Legitimate programs rarely set memory as RWX

Our Most Common Execution Methods



Portable **Executables** (EXE)

- Various execution methods
- Easy to blend in with other legitimate processes
- Highly scrutinized
- Can lead to detection by filetype execution alone
- Can be outright blocked with application allow-listing
- ...and yet, when done right, this is one of the most common file types we execute

EXE Execution

- CMD.exe
- Explorer.exe
 - Can break the parent-child process tree with /separate flag
- MSI Installer File
 - Can lead to privilege escalation in the right circumstances
- Shortcut (LNK) Files
 - Can hide the file extension in the right social engineering scenarios
- Service Binaries

LOLBAS - MSBuild

- .NET Compiler
- Included with Windows 10 by default
- Can execute C# code within an XML file as an inline task
 - Execution command: MSBuild.exe payload.xml
- Application allow-listing bypass since it is signed by Microsoft
- XML payload file can bypass static detections due to text nature of file

LOLBAS - MSBuild

- Payload file extension can be changed from XML to C# Project file (CSPROJ) for automatic execution of payload
 - Must be in the same directory as MSBuild.exe
 - Copy MSBuild.exe and payload file to the same directory and use as persistence
 - No arguments are recorded since payload is automatically parsed
- Can load the C# code from remote file hosted on WebDAV
- Full Windows API calls accessible through C# code

LOLBAS - InstallUtil

- .NET Install and Uninstall utility
- Included with Windows 10 by default
- Application allow-listing bypass since it is signed by Microsoft
- Can execute C# code compiled in DLL form
 - May require dropping DLL to disk
 - Potential indicator of compromise

LOLBAS - InstallUtil

- Execution command: `InstallUtil.exe /u payload.dll`
- Build execution into social engineering pretext
 - Hide DLL amongst other files in container such as ISO or Zip
 - Create LNK file with command needed to execute

Proxy DLL

- Export legitimate DLL function names
- Proxy function requests to legitimate DLL
 - Example: C:\Windows\System32\version.dll
- Payload executes each time the DLL function is called
- Legitimate Microsoft functionality
- Similar to DLL hijacking but allows program to run without error
- Spartan Proxy DLL Generator

Proxy DLL Attack Scenario

- Drop in a directory with legitimate executable
 - Program folders installed in User's Appdata folder are writable by user
- DLL search order finds proxy DLL first
- Function calls are proxied to legitimate DLL
- Payload is loaded into calling process
- Attack chain is broken if dropped by initial access payload and later executed by different binary



Conclusion

Training

Free

- 0xPat Blog (<https://0xpat.github.io>)
- Cocomelonc Blog (<https://cocomelonc.github.io/>)

Paid

- Sektor7 Malware Development Courses
- MalDev Academy
- Red Siege Intrusion Ops course
- White Knight Labs Offensive Development course
- ZeroPoint Security Red Team Ops course

Resources

Free

- Donut (<https://github.com/TheWover/donut>)
- UPX Packer (<https://upx.github.io>)
- Freeze (<https://github.com/tylous/Freeze>)
- DarkArmour (<https://github.com/bats3c/darkarmour>)
- ProtectMyTooling (<https://github.com/mgeeky/ProtectMyTooling>)
- ConfuserEx (<https://github.com/mkaring/ConfuserEx>)

Paid

- Balliskit ShellcodePackPro/MacroPackPro (<https://balliskit.com>)

Takeaways

- Malware development is an entire field of it's own
- This talk is an inch deep and a mile wide
- The methods described here are meant as jumping off points
- Self-study and research will be necessary to use these techniques
- Understanding the method can help with understanding other protection methodologies and how they are bypassed

Questions?

- Reach out on Discord @covers
- <https://redsiege.com/discord>
- <https://redsiege.com/wedoff>

Slides?

- <https://redsiege.com/payload>



BUSINESS:
GETOFFENSIVE@REDSIEGE.COM

OFFENSIVE SERVICES. OFFENSIVE MINDS



**ASSUMED BREACH
ASSESSMENT**



**PENETRATION
TESTING**



**WEB APPLICATION
PENETRATION TESTING**



**RANSOMWARE
READINESS ASSESSMENT**



**RED TEAM &
ADVERSARY EMULATION**



**PURPLE TEAM &
TRAINING**



OUR OFFENSE PREPARES YOUR DEFENSE